

Classification

Rita Almeida

Autumn 2025

We start by loading the libraries needed in this lab. If you do not have these packages installed you must start by doing this. For example if you do not have the package *MASS*, you must run the command `install.packages("MASS")`.

```
library('tidyverse')
library("gridExtra")
library("MASS")
library("class")
library("ISLR")
theme_set(theme_bw())
```

Logistic regression

We will use a data set from a gesture perception experiment, which is described in the course book *Statistics for Linguists, an introduction using R, from B Winter* and also in the article Hassemer, J., & Winter, B. (2016) *Producing and perceiving gestures conveying height or shape*. *Gesture*, 15(3), 404–424. In short, the authors collected information about how onlookers inferred information depending on hand gestures. Their hypothesis was that when the pinkie finger, the ring finger, and the middle finger are curled in, the distance between the index finger and the thumb pad are interpreted as *height* of an object. In opposition, when the fingers are extended the interpretation is more likely to be that the C-shape formed by the index finger and thumb refer to the *shape* on an object. To test this hypothesis the authors showed a group of participants pictures of a single hand shape, where the fingers were curled to different degrees in a continuum from most extended (1) to most curved (9). There were pictures with 9 positions in the continuum. The task of the subjects was to choose *height* or *shape* for the picture shown, according to how they interpreted the gesture to be indicative of the size or shape of an object.

Data visalization

The data is on the file `hassemer_winter_2016_gesture.csv`. We will start by loading the file and visualize the first rows.

```
ges <- read_csv('hassemer_winter_2016_gesture.csv')

## Rows: 309 Columns: 5
## -- Column specification -----
## Delimiter: ","
## chr (2): question_order, choice
## dbl (3): index_curve, pinkie_curl, confidence
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(ges, 10)
```

```
## # A tibble: 10 x 5
##   index_curve pinkie_curl question_order confidence choice
##   <dbl>      <dbl> <chr>          <dbl> <chr>
## 1         1         9 height_first      8 height
## 2         5         6 shape_first       7 shape
## 3         2         7 height_first      8 height
## 4         4         3 height_first      8 shape
## 5         1         1 height_first      8 shape
## 6         6         9 height_first      9 shape
## 7         6         7 shape_first       6 shape
## 8         3         7 shape_first       7 shape
## 9         1         2 shape_first       3 shape
## 10        6         2 height_first       7 shape
```

The data set *ges* contains several variables. Here we will use the following:

choice, categorical variable containing the choice of the subject for a given picture.

pinkie_curl, variable taking values from 1 to 9, indicating the curvature of the pinkie, ring and middle fingers.

index_curve, variable taking values from 1 to 6, indicating the curvature of the index finger.

To get an impression of the data, one can look at a table of the distribution of the values of the variable *pinkie_curl*. That is, how many times each picture was used in the experiment. Note that this is the main variable which the authors thought to be important for the information inferred by the subjects.

```
table(ges$pinkie_curl)
```

```
##
##  1  2  3  4  5  6  7  8  9
## 33 37 37 30 42 44 32 24 30
```

Next, we look at the counts on the two possible choices made by the subjects.

```
table(ges$choice)
```

```
##
## height  shape
##   125    184
```

The same table can be shown in terms of proportion of the choices, instead of the actual counts, using the command *prop.table()*. The proportion table can give a better intuition of the data.

```
prop.table(table(ges$choice))
```

```
##
## height  shape
## 0.4045307 0.5954693
```

Finally, we look at a table with the counts of *height* and *shape* for each value of *pinkie_curl*. It seems that when the fingers are more curved (*pinkie_curl* 8 or 9) there are more choices of *height* than *shape*, as one might expect.

```
xtab <- table(ges$pinkie_curl, ges$choice)
xtab
```

```
##
## height shape
## 1     14    19
```

```
## 2    17    20
## 3    12    25
## 4     5    25
## 5     9    33
## 6    15    29
## 7    15    17
## 8    18     6
## 9    20    10
```

The same table can be displayed in terms of proportions.

```
round(prop.table(xtab, 1), digits = 2)
```

```
##
##      height shape
## 1  0.42 0.58
## 2  0.46 0.54
## 3  0.32 0.68
## 4  0.17 0.83
## 5  0.21 0.79
## 6  0.34 0.66
## 7  0.47 0.53
## 8  0.75 0.25
## 9  0.67 0.33
```

Logistic regression with one explanatory variable

Our first task is to see if the variable *pinkie_curl* can be used to classify the choices into *shape* or *height*. To do this we start by telling R that *choice* is a categorical variable by converting *choice* into a factor, with *shape* as the baseline level. Then we use the command *contrast()* to check that indeed *shape* corresponds 0 and *height* to 1.

```
ges$choice <- factor(ges$choice, levels = c("shape", "height"))
contrasts(ges$choice)
```

```
##      height
## shape      0
## height     1
```

Next we divide the data into two sets: the training data set *ges.Train* with 209 observations, and the test data set *ges.Test* with 100 observations.

```
ges.Train <- ges[1: 209, ]
ges.Test  <- ges[210: 309, ]
```

We fit a logistic model on the training data, using as only explanatory variable *pinkie_curl*. To fit a logistic regression we use the command *glm()*, which has a similar syntax to the command *lm()* we used to fit linear regression models. The only difference is that to use *glm()* we have to give the argument *family='binomial'* to indicate that we want to fit a logistic model. After fitting the model we can look at the estimated coefficients using the *summary()**\$coef* command.

```
ges_m <- glm(choice ~ pinkie_curl, data = ges.Train,
             family = 'binomial')
summary(ges_m)$coef
```

```
##              Estimate Std. Error  z value    Pr(>|z|)
## (Intercept) -0.8363988 0.31233747 -2.677869 0.007409222
## pinkie_curl  0.1053986 0.05608675  1.879207 0.060216238
```

The results show that the log odds of observing a *height* response increase by 0.105 for each step along the *pinkie_curl* variable. We next use the function *predict()* to generate the estimated logistic regression curve. We use *type="response"* to generate probabilities, instead of log odds, for each value of *pinkie_curl*.

```
ges_preds <- tibble(pinkie_curl = 1:9)
ges_preds$probs <- predict(ges_m, ges_preds, type="response")
ges_preds$probs
```

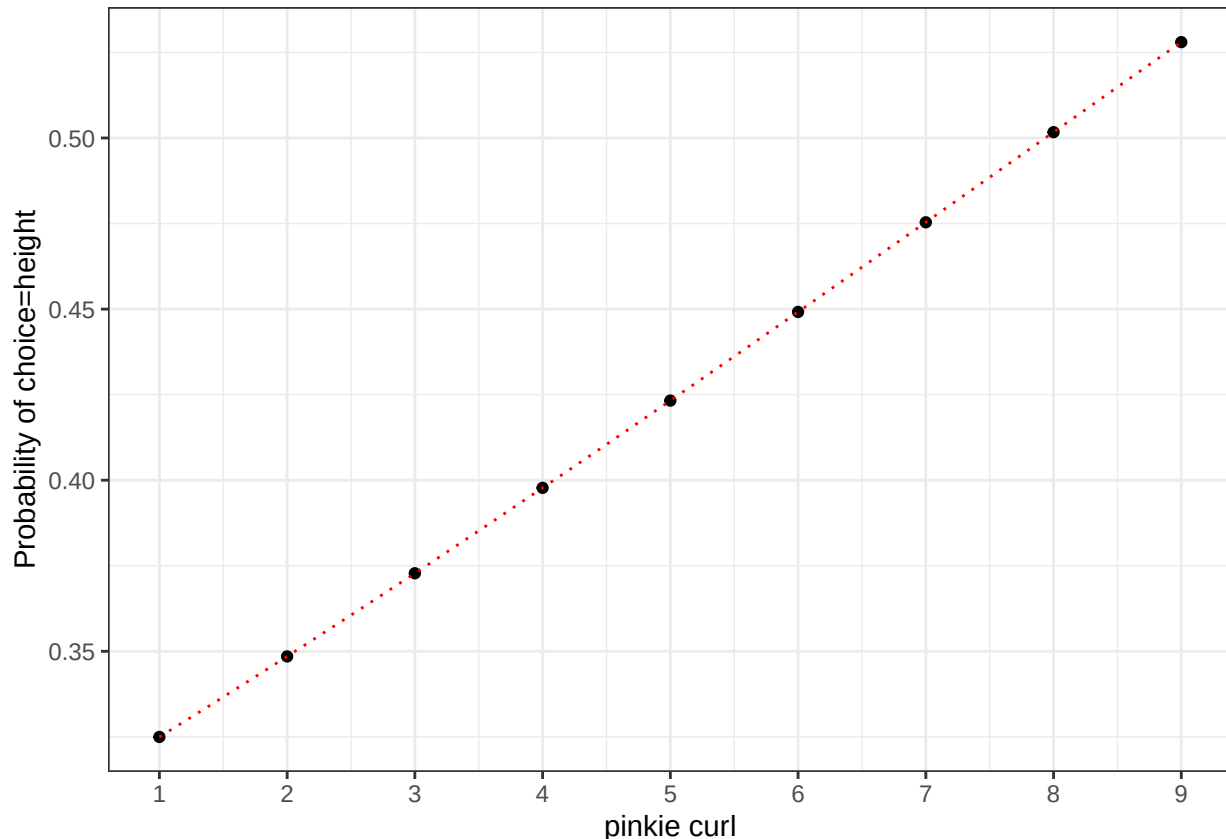
```
##          1          2          3          4          5          6          7          8
## 0.3249753 0.3485085 0.3728048 0.3977607 0.4232598 0.4491744 0.4753678 0.5016975
##          9
## 0.5280178
```

Finally we plot the probabilities as a function of *pinkie_curl*. We start by generating probabilities for intermediate levels of the explanatory variable, to generate the estimated regression function shown in the red dotted line.

```
ges_preds2 <- tibble(pinkie_curl = seq(1, 9, by=0.01))
ges_preds2$probs <- predict(ges_m, ges_preds2, type="response")
```

And then we generate the plot.

```
ggplot() +
  geom_point(aes(x = ges_preds$pinkie_curl, y = ges_preds$probs)) +
  geom_line(aes(x = ges_preds2$pinkie_curl, y = ges_preds2$probs), color="red", linetype="dotted") +
  xlab("pinkie curl") + ylab("Probability of choice=height") + scale_x_continuous(breaks = 1:9)
```



Now that we have a model fitted using the training data set, we can use it to make a classification of the data on the test data set. We start by predicting the probabilities for the test set. (In this case the *pinkie_curl*

variable only takes 9 values, so we actually already had these probabilities.)

```
ges.Test$probs <- predict(ges_m, ges.Test, type="response")
```

Then we have to generate the *choices* (*height* or *shape*) according to the fitted model. We attribute *height* to *choice* if the probabilities are larger than 0.5 according to our fitted model. We attribute *shape* to the variable *choice* otherwise. We then display a confusion matrix and calculate the fraction of correct classifications and incorrect classifications.

```
ges.Test$preds <- rep("shape", 100)
ges.Test$preds[ges.Test$probs > 0.5] = "height"
table(ges.Test$preds, ges.Test$choice)
```

```
##
##           shape height
## height      3      12
## shape      60      25
```

```
mean(ges.Test$preds == ges.Test$choice)
```

```
## [1] 0.72
```

```
mean(ges.Test$preds != ges.Test$choice)
```

```
## [1] 0.28
```

We can see that the model correctly predicted the *choice* of *shape* in 60 cases and correctly predicted the choice of *height* in 12 cases. However, the model predicted *shape* in 25 cases where the subjects chose *height*. The error rate on the test data was 28%, and the accuracy 72%.

Logistic regression using several explanatory variables

In this section we use more explanatory variables in the model. In particular we want to use the variable *index_curve* and a quadratic term of the curving of the pinkie, ring and middle fingers. For this we start by creating a variable that is the square of the *pinkie_curl*, after mean centering, and call it *pinkie_sq*.

```
ges$pinkie_sq <- (ges$pinkie_curl - mean(ges$pinkie_curl))^2
```

Next we divide the data into new training and test sets, *ges.Train2* and *ges.Test2*, respectively.

```
ges.Train2 <- ges[1: 209, ]
ges.Test2 <- ges[210: 309, ]
```

Then we fit a new logistic regression model with the three explanatory variables mentioned: *pinkie_curl*, *pinkie_sq* and *index_curve*.

```
ges_m2 <- glm(choice ~ pinkie_curl + pinkie_sq + index_curve, data = ges.Train2,
              family = 'binomial')
summary(ges_m2)$coef
```

```
##           Estimate Std. Error    z value    Pr(>|z|)
## (Intercept)  0.19289282 0.47088129  0.4096421 6.820685e-01
## pinkie_curl  0.12414827 0.06214128  1.9978388 4.573413e-02
## pinkie_sq    0.09819414 0.02842937  3.4539683 5.524024e-04
## index_curve -0.49820659 0.10193936 -4.8872839 1.022366e-06
```

Looking at the estimate log odds, it seems that all three variables contribute to the choice.

We proceed by applying the new fitted model to the test data. Next we follow the steps above to display a confusion matrix, and calculate the fraction of correct classifications and incorrect classifications.

```
ges.Test2$probs <- predict(ges_m2, ges.Test2, type="response")
```

```
ges.Test2$preds <- rep("shape", 100)
ges.Test2$preds[ges.Test2$probs > 0.5] = "height"
table(ges.Test2$preds, ges.Test2$choice)
```

```
##
##           shape height
## height      12      26
## shape       51      11
```

```
mean(ges.Test2$preds == ges.Test2$choice)
```

```
## [1] 0.77
```

```
mean(ges.Test2$preds != ges.Test2$choice)
```

```
## [1] 0.23
```

The classification accuracy is better with this model, reaching 77% correct predictions. In particular, this model shows increase correct classification of the *height* choices.

Linear discriminant analysis

In this section we will apply linear discriminant analysis (LDA) to the same data set we used above. One can do this in R by using the `lda()` command, which is part of the library *MASS* and works in the same way as the `lm()` command we have used to fit linear regressions. We use for the classification the same variables we used above, that is *pinkie_curl*, *pinkie_sq* and *index_curve*. We start by fitting the model using the training data set.

```
ges_m3 <- lda(choice ~ pinkie_curl + pinkie_sq + index_curve, data = ges.Train2)
```

And then we check the model:

```
ges_m3
```

```
## Call:
## lda(choice ~ pinkie_curl + pinkie_sq + index_curve, data = ges.Train2)
##
## Prior probabilities of groups:
##      shape      height
## 0.5789474 0.4210526
##
## Group means:
##      pinkie_curl pinkie_sq index_curve
## shape      4.578512  5.242023   4.082645
## height      5.250000  8.076460   2.920455
##
## Coefficients of linear discriminants:
##              LD1
## pinkie_curl  0.1230453
## pinkie_sq    0.1003753
## index_curve -0.5224447
```

The output shows that $\hat{\pi}_s = 0.57895$ and $\hat{\pi}_h = 0.42105$, that is $\approx 58\%$ of the training choices were *shape* and $\approx 42\%$ were *height*.

It also shows the group means which are used as estimators of μ_k . From these means one sees that it seems that a choice of *height* is associated with higher values of *pinkie_curl* and *pinkie_sq*, but smaller values of *index_curve*.

The coefficients of linear discriminants give the linear combination of the predictors which is used for the LDA classification. If $0.1230453 \times \text{pinkie_curl} + 0.1003753 \times \text{pinkie_sq} - 0.5224447 \times \text{index_curve}$ is large then the LDA classifies the choice as *height* and if it is low as *shape*.

Now that we have the linear discriminant, we can apply it to the test data set to check the accuracy of the model for new data.

```
ges_m3.pred <- predict(ges_m3, ges.Test2)
```

The output *ges_m3.pred* is a list with three elements:

ges_m3.pred\$class which contains the classification results (*shape* or *height*) for each observation.

posterior that contains the posterior probabilities $\Pr(Y = K|X = x) = \frac{\pi_k f_k(x)}{\sum_{l=1}^K \pi_l f_l(x)}$

x that contains the linear discriminants

```
head(ges_m3.pred$class, 10)
```

```
## [1] height shape height shape shape shape shape shape height
## Levels: shape height
```

```
head(ges_m3.pred$posterior, 10)
```

```
##      shape      height
## 1  0.4342958 0.5657042
## 2  0.8578648 0.1421352
## 3  0.4406742 0.5593258
## 4  0.5608662 0.4391338
## 5  0.8358945 0.1641055
## 6  0.6799847 0.3200153
## 7  0.8134339 0.1865661
## 8  0.5252093 0.4747907
## 9  0.6686008 0.3313992
## 10 0.4342958 0.5657042
```

```
head(ges_m3.pred$x, 10)
```

```
##      LD1
## 1  0.6750847
## 2 -1.4412964
## 3  0.6484824
## 4  0.1526400
## 5 -1.2671481
## 6 -0.3698045
## 7 -1.1075336
## 8  0.3001859
## 9 -0.3165997
## 10 0.6750847
```

Once we have the classification, we can generate a confusion matrix, as before.

```
LDA.pred <- (ges_m3.pred$class)
table(LDA.pred, ges.Test2$choice)
```

```
##
```

```
## LDA.pred shape height
##   shape      51      12
##   height     12      25
```

We can also calculate the fraction of correct classifications and incorrect classifications.

```
mean(LDA.pred==ges.Test2$choice)
```

```
## [1] 0.76
```

```
mean(LDA.pred!=ges.Test2$choice)
```

```
## [1] 0.24
```

One could also generate the predictions based on the posterior probabilities and predict *height* every time the corresponding posterior is larger or equal to 0.5.

```
LDA.pred2 <- (ges_m3.pred$posterior[,2]>=0.5)
table(LDA.pred2, ges.Test2$choice)
```

```
##
## LDA.pred2 shape height
##   FALSE     51     12
##   TRUE      12     25
```

Exercise

Imagine that one would want to do the same classification, but use a less strict threshold to predict the choice to be *height*. For example one could predict the choice to be *height* for a posterior larger or equal to 0.4. Generate the confusion matrix for this case. Now repeat the procedure for an even less strict threshold of 0.3 and generate the respective confusion matrix.

How does the number of true positives and false positives compare with those calculated with the threshold above?

```
#write your solution here
```

K-nearest neighbors

To perform a k-nearest neighbors classification we use the function *knn()* which is a part of the library *class*. The function *knn()* does not work in the same way as the regression functions we used before. *knn* needs four different inputs:

1. A matrix containing the predictors for the training data set.
2. A matrix containing the predictors for the test data set.
3. A vector containing the output (class labels) for the training data set.
4. A value for *k* to use in the algorithm.

This section follows the lab in the course book James et al. *An introduction to statistical learning*. We use the data set *Caravan* which is part of the library *ISLR*. The data set is from an insurance company and contains observations of 86 variables for 5822 individuals. The outcome or response variable we want to use is *Purchase*, which takes the value *Yes* for individuals that have purchased a caravan insurance policy and *No* for the others. Variables 1 to 43 contain sociodemographic data and the variables 44 to 86 data about product ownership. *Purchase* is the variable 86 in the data set.

We start by looking at the proportion of individuals that have purchased a caravan insurance policy.


```
round(prop.table(table(Caravan$Purchase)),digits=4)
```

```
##  
##      No      Yes  
## 0.9402 0.0598
```

We see that less than 6% of the individuals have the insurance, so the outcome variable is very unbalanced.

The KNN algorithm involves computing the distance between observations. When the different variables have very different scales this poses a problem. For the algorithm, a salary difference of 1000 dollars is much larger, and hence more determinant, than an age difference of 50 years. To avoid this problem, one standardizes the variables before doing the classification. That is for each variable, one subtracts the variable mean from each observation, and divides it by the standard deviation. In this way all variables are scaled to have mean 0 and standard deviation 1.

We start by standardizing the data, after excluding the last variable, which is the variable *Purchase*, our categorical outcome variable.

```
stand.X <- scale(Caravan[,-86])
```

We can verify with the first variable of the data set that it is now standardized. Before standardization the mean and standard deviation of this variable were:

```
mean(Caravan[,1])
```

```
## [1] 24.25335
```

```
sd(Caravan[,1])
```

```
## [1] 12.84671
```

After the standardization, the same variable has the following mean and standard deviation:

```
mean(stand.X[,1])
```

```
## [1] -7.025576e-17
```

```
sd(stand.X[,1])
```

```
## [1] 1
```

Now we split the data set into a test set containing the first 1000 observations, and a training test containing the rest of the observations.

```
test.X <- stand.X[1:1000,]  
train.X <-stand.X[1001:5822, ]
```

Then, we create vectors with the outcome variable *Purchase* for the test set and for the training set.

```
test.Y <- Caravan$Purchase[1:1000]  
train.Y <-Caravan$Purchase[1001:5822]
```

Now we have all the inputs to *knn()*, and we train the model considering $k = 1$.

```
set.seed(1)  
knn.pred <- knn(train.X,test.X,train.Y,k=1)
```

To check the accuracy of the predictions we can do as before:

```
mean(test.Y==knn.pred)
```

```
## [1] 0.882
```

```
mean(test.Y!=knn.pred)
```

```
## [1] 0.118
```

We see that KNN model gives a correct prediction in more than 88% of the cases. The error rate is smaller than 12%. This should be interpreted with some care, given that the outcome variable *Purchase* was so unbalanced. If we had a classifier that just predicts *Purchase=No* independently of the observations, the classifier would have an error rate of 6%, since only 6% of the individuals have purchased an insurance! To understand the results and evaluate if this classifier is useful, it is helpful to look at the confusion table. The table can be obtained using the same code as before:

```
table(knn.pred,test.Y)
```

```
##           test.Y
## knn.pred  No  Yes
##      No   873  50
##      Yes   68   9
```

If the company would have tried to sell insurance policies to a random selection of individuals, they would have 6% of success rate. However, if one would try to sell it to the 77 individuals that the KKN model predicts will buy the insurance, then the success rate would be 11.7%, ($0.117 = 9/(68 + 9)$). This is double the rate that would have been obtained with random guessing!

Exercise

Try the KNN classification, but now using $k=3$ and then $k=5$. Comment the results obtained.

```
#write your solution here
```

Exercise

Use the *Caravan* data to perform the same classification, but this time using LDA. Generate the predictions based on the posterior probabilities with a threshold of 0.5 and then with a threshold of 0.25. Comment the results.

```
#write your solution here
```